OWASP Authentication Cheat Sheet

1. Introduction

Authentication (AuthN)

Authentication is the process of verifying that an individual, entity, or website is who or what it claims to be. It determines the validity of one or more authenticators, such as:

- Passwords
- Fingerprints
- Security tokens

Digital Identity

Digital Identity is the unique representation of a subject engaged in an online transaction.

A digital identity is always unique within the context of a digital service but does not necessarily need to be traceable back to a specific real-life subject.

Identity Proofing

Identity Proofing establishes that a subject is actually who they claim to be.

- Related to KYC (Know Your Customer) concepts.
- Aims to bind a digital identity with a real person.

Session Management

Session Management is the process by which a server maintains the state of an entity interacting with it.

- Allows the server to remember how to react to subsequent requests throughout a transaction.
- Sessions are maintained on the server using a **Session Identifier**.
- The session identifier is passed between the client and server when sending and receiving requests.
- Session identifiers should be unique per user and computationally very difficult to predict.

---

2. Authentication General Guidelines

2.1 User IDs

The primary function of a User ID is to uniquely identify a user within a system.

Recommendations:

- Ideally, User IDs should be randomly generated.
- Avoid predictable or sequential IDs.
- Predictable IDs can create security risks, especially when IDs are exposed or inferred from external sources.

2.2 Usernames

Usernames are easy-to-remember identifiers chosen by users to identify themselves when logging into a system or service.

- User ID and username may be used interchangeably if the username also serves as the user's unique identifier.
- Users should be allowed to use their email address as a username, provided the email is verified during sign-up.
- Users should also have the option to choose a username other than an email address.

2.3 Authentication Solutions and Sensitive Accounts

- **Do NOT** allow login using sensitive accounts through any front-end user interface.
    - Sensitive accounts include accounts used internally by the solution, such as backend, middleware, or database accounts.
- **Do NOT** use the same authentication solution (e.g., IDP / AD) used internally for unsecured access, such as public access or a DMZ.

2.4 Implement Proper Password Strength Controls

A strong password policy makes it difficult or improbable for attackers to guess passwords through manual or automated methods.

A. Password Length

- Applications should enforce a minimum password length.
- If MFA is enabled, passwords shorter than **8 characters** are considered weak.
- If MFA is not enabled, passwords shorter than **15 characters** are considered weak.
- Maximum password length should be **at least 64 characters** to support passphrases.
- Be aware that certain password-hashing implementations may cause long-password denial-of-service vulnerabilities.

Why are longer passphrases effective?

- Longer passphrases increase the number of guesses an attacker's dictionary or wordlist must cover.
- Their effectiveness does not depend on a precise entropy value.
- NIST notes that estimating entropy for user-chosen passwords is challenging.
- Password length and blocklist checks are preferred over composition rules or entropy calculations.
- Entropy estimates depend on assumptions about the search space and should be treated as illustrative, not absolute measures of password strength.

B. Common Password Attacks

Attackers commonly use:

- Common-password dictionaries and password lists.
- Previously compromised credentials.
- Credential stuffing attacks.

These techniques are reflected in MITRE ATT&CK T1110 – Brute Force, which includes:

- Password Guessing
- Password Spraying
- Credential Stuffing

Screening passwords against blocklists helps prevent users from selecting passwords that are commonly used or already known to attackers.

C. Additional Password Recommendations

- Do not silently truncate passwords.
- Allow all characters, including Unicode and whitespace.
- Do not impose password composition rules that restrict character types.
- Do not require uppercase letters, lowercase letters, numbers, or special characters.
- Ensure credential rotation when a password leak or compromise is identified, or when authenticator technology changes.
- Avoid requiring periodic password changes.
- Encourage users to choose strong passwords and enable MFA.

D. Password Strength Meter

Include a password strength meter to help users create more complex passwords.

- The **zxcvbn-ts library** can be used for this purpose.
- Other language implementations of zxcvbn are also available.
- Check the age and maturity of each implementation before using it.

E. Block Common and Previously Breached Passwords

- Use **Pwned Passwords** to check passwords against previously breached passwords.
- The Pwned Passwords database can also be downloaded and hosted locally.
- Other password lists are available through resources such as SecLists, although their update frequency is not guaranteed.

For More Detailed Information

- ASVS v5.0 Password Security Requirements
- Passwords Evolved: Authentication Guidance for the Modern Era

2.5 Implement a Secure Password Recovery Mechanism

Applications commonly provide a mechanism that allows users to regain access to their accounts if they forget their passwords.

A secure password recovery mechanism should be implemented.

2.6 Store Passwords Securely

It is critical to store passwords using the correct cryptographic technique.

Refer to the OWASP Password Storage Cheat Sheet for detailed guidance.

2.7 Compare Password Hashes Using Safe Functions

Where possible, compare the user-supplied password with the stored password hash using a secure password comparison function provided by the language or framework.

Example: PHP `password_verify()`.

If a secure built-in function is unavailable, ensure that the comparison function:

1. Has a maximum input length to protect against denial-of-service attacks using very long inputs.
2. Explicitly sets the type of both variables to protect against type confusion attacks, such as Magic Hashes in PHP.
3. Returns in constant time to protect against timing attacks.

2.8 Change Password Feature

When developing a change-password feature, ensure that:

- The user is authenticated with an active session.
- The current password is verified before allowing the password change.

Why is current password verification important?

Consider a situation where:

- A user logs into a public computer.
- The user forgets to log out.
- Another person accesses the active session.

Without current password verification, the other person might be able to change the account password.

2.9 Transmit Passwords Only Over TLS or Other Strong Transport

The login page and all subsequent authenticated pages must be accessed exclusively over TLS or another strong transport mechanism.

Risks of not using TLS:

- If the login page does not use TLS, an attacker may modify the login form action and cause credentials to be sent to an arbitrary location.
- If authenticated pages do not use TLS, an attacker may view the unencrypted session ID and compromise the user's authenticated session.

2.10 Require Re-authentication for Sensitive Features

Require users to provide their current credentials before performing sensitive operations, such as:

- Updating their password.
- Changing their email address.
- Shipping a purchase to a new address.
- Performing other sensitive transactions.

Why is this necessary?

Without re-authentication:

- An attacker may execute sensitive transactions through CSRF or XSS attacks without knowing the user's current credentials.
- An attacker may gain temporary physical access to the user's browser.
- An attacker may steal the user's session ID and take over the session.

2.11 Re-authentication After Risk Events

Re-authentication is critical when an account experiences high-risk activity, such as account recovery, password resets, or suspicious behavior.

A. When to Trigger Re-authentication

- **Suspicious Account Activity:** Unusual login patterns, IP address changes, or new device enrollments.
- **Account Recovery:** After users reset passwords or change sensitive account details.
- **Critical Actions:** High-risk actions such as changing payment details or adding new trusted devices.

B. Re-authentication Mechanisms

1. **Adaptive Authentication**
    - Use risk-based authentication models that adapt to user behavior and context.
2. **Multi-Factor Authentication (MFA)**
    - Require an additional layer of verification for sensitive actions or events.
3. **Bound Authenticators**
    - Require an authenticator already bound to the account, such as the current password or a passkey.
    - Do not substitute security questions.

C. Implementation Recommendations

- **Minimize User Friction:** Avoid unnecessarily disrupting the user experience.
- **Context-Aware Decisions:** Base re-authentication decisions on context, such as geolocation, device type, and previous patterns.
- **Secure Session Management:** Invalidate sessions after re-authentication and rotate tokens.

2.12 Consider Strong Transaction Authentication

Some applications should use a second authentication factor to verify whether a user is allowed to perform sensitive operations.

This provides an additional layer of security for important transactions.

2.13 TLS Client Authentication

TLS Client Authentication, also known as **Two-Way TLS Authentication**, involves both the browser and server sending their respective TLS certificates during the TLS handshake.

How it works:

1. The browser and server exchange TLS certificates during the handshake.
2. The server validates the client's certificate using a trusted third-party CA or its own CA.
3. The server authenticates the user based on the certificate.
4. The user installs a certificate generated specifically for them in their browser.
5. The certificate contains subject values that help identify the user.

When is TLS Client Authentication Appropriate?

- When users should access the website only from a single computer or browser.
- When users are comfortable installing TLS certificates, or IT support can assist them.
- When the website requires an additional security step.
- When the website is an organization's intranet.

Limitations

- It is generally not suitable for widely available public websites with average users.
- Certificate installation can be difficult for ordinary users.
- It can prevent users from having to type passwords, protecting against average keyloggers.
- Combining a password with TLS client authentication may provide additional security.
- If the client is behind an enterprise proxy performing SSL/TLS decryption, certificate authentication may break unless the website is allowed on the proxy.

---

3. Authentication and Error Messages

Incorrectly implemented authentication error messages can allow attackers to perform **User ID and Password Enumeration**.

Applications should return generic responses in both HTTP and HTML.

3.1 Authentication Responses

For login, password reset, and password recovery, return a generic error message regardless of whether:

- The user ID or password is incorrect.
- The account does not exist.
- The account is locked or disabled.

The same approach should be considered for account registration when the user already exists.

Objective: Prevent attackers from discovering valid accounts through discrepancies in application responses.

3.2 Timing-Based Attacks

Even if the error message is generic, the time taken to process a request may reveal information.

For example:

- If a username does not exist, the application may immediately return an error.
- If the username exists but the password is incorrect, the application may perform additional processing before returning an error.

This difference in processing time can allow attackers to distinguish between invalid usernames and incorrect passwords.

3.3 Example: Login Implementation

First Implementation – Quick Exit Approach

```
IF USER_EXISTS(username) THEN
   password_hash = HASH(password)
   IS_VALID = LOOKUP_CREDENTIALS_IN_STORE(username, password_hash)

   IF NOT IS_VALID THEN
      RETURN Error("Invalid Username or Password!")
   ENDIF
ELSE
   RETURN Error("Invalid Username or Password!")
ENDIF
```

Problem:

- If the user does not exist, the application immediately returns an error.
- If the user exists but the password is incorrect, additional processing takes place.
- The response times differ.
- Attackers can use this timing difference to identify valid usernames.

Second Implementation – Without Quick Exit

```
password_hash = HASH(password)
IS_VALID = LOOKUP_CREDENTIALS_IN_STORE(username, password_hash)

IF NOT IS_VALID THEN
   RETURN Error("Invalid Username or Password!")
ENDIF
```

Advantage:

- The application follows approximately the same process regardless of whether the username exists.
- This helps ensure approximately equal response times.
- It makes it harder for attackers to distinguish between an invalid username and an incorrect password.

3.4 User Experience Considerations

Generic error messages can sometimes confuse legitimate users.

For example:

- Users may not understand why they cannot log in.
- Repeated unsuccessful attempts may frustrate users.
- Users may eventually leave the application.

The decision to use generic error messages should depend on the criticality of the application and its data.

For critical applications, the team may redirect users to a support page while displaying a generic error message.

Protection against brute-force attacks also helps prevent user enumeration at scale.

CAPTCHA can be used when a generic error message cannot be returned because the user experience must be preserved.

3.5 Incorrect and Correct Response Examples

A. Login

Incorrect responses:

- "Login for User foo: invalid password."
- "Login failed, invalid user ID."
- "Login failed; account disabled."
- "Login failed; this user is not active."

Correct response:

- "Login failed; Invalid user ID or password."

B. Password Recovery

Incorrect responses:

- "We just sent you a password reset link."
- "This email address doesn't exist in our database."

Correct response:

- "If that email address is in our database, we will send you an email to reset your password."

C. Account Creation

Incorrect responses:

- "This user ID is already in use."
- "Welcome! You have signed up successfully."

Correct response:

- "A link to activate your account has been emailed to the address provided."

3.6 Error Codes and URLs

An application may return different HTTP error codes depending on the authentication result.

For example:

- HTTP 200 for a successful result.
- HTTP 403 for a failed result.

Even if the application displays a generic error page, different HTTP response codes may reveal whether an account is valid.

Error disclosure can therefore become another discrepancy factor.

---

4. Protect Against Automated Attacks

Attackers use automated techniques to compromise user accounts.

4.1 Common Automated Attacks

Attack Type	Description
Brute Force	Testing multiple passwords from a dictionary or another source against a single account.
Credential Stuffing	Testing username/password combinations obtained from breaches of other websites.
Password Spraying	Testing a single weak password against a large number of different accounts.

4.2 Defense-in-Depth Approach

Different protection mechanisms can be implemented against automated attacks.

- Individual defenses may not provide complete protection.
- Combining multiple defenses through a **Defense-in-Depth** approach provides a reasonable level of protection.
- The following controls primarily focus on preventing brute-force attacks but may also help against other attack types.

4.3 Multi-Factor Authentication (MFA)

MFA is one of the strongest defenses against most password-related attacks, including brute-force attacks.

- Microsoft analysis suggests MFA would have stopped 99.9% of account compromises.
- MFA should be implemented wherever possible.
- Depending on the application's audience, enforcing MFA may not always be practical or feasible.

4.4 Login Throttling

Login Throttling is a security mechanism that prevents attackers from making too many password-guessing attempts through normal interactive means.

It includes controls such as:

- Maximum number of login attempts.

Account Lockout

Account lockout is a common protection mechanism against automated attacks.

It prevents further login attempts for a period after a certain number of failed login attempts.

Important recommendations:

- Associate the failed-login counter with the account itself rather than the source IP address.
- This prevents attackers from bypassing the limit by using multiple IP addresses.

Consider the following factors when designing an account lockout policy:

1. **Lockout Threshold:** Number of failed attempts before the account is locked.
2. **Observation Window:** Time period within which failed attempts must occur.
3. **Lockout Duration:** How long the account remains locked.

Exponential Lockout

Instead of using a fixed lockout duration, some applications use exponential lockout.

- The lockout duration starts with a very short period, such as one second.
- The duration doubles after each failed login attempt.

The source also mentions delaying after each account lockout, with a maximum of 2–3 attempts before permanent account lockout.

Preventing Denial-of-Service Attacks:

Care must be taken to ensure that attackers cannot deliberately lock out other users' accounts.

One possible approach is to allow users to use the forgotten-password functionality to log in, even if their accounts are locked out.

4.5 CAPTCHA

An effective CAPTCHA can help prevent automated login attempts.

However:

- Some CAPTCHA implementations have weaknesses.
- Automated techniques may solve them.
- CAPTCHA-solving services can also be used by attackers.

Therefore, CAPTCHA should be treated as a **Defense-in-Depth control**, making brute-force attacks more time-consuming and expensive rather than completely preventing them.

Usability recommendation:

- It may be more user-friendly to require CAPTCHA only after a small number of failed login attempts.
- Avoid requiring CAPTCHA from the very first login.

4.6 Security Questions and Memorable Words

- Do not use security questions for authentication or re-authentication.
- Knowledge-based authentication is excluded by ASVS 6.4.2.
- When verifying a password or memorable word, request and verify the **full secret**, rather than selected characters.

Security questions may have a limited role in legacy systems, but should not be used as a modern authentication factor.

---

5. Logging and Monitoring

Enable logging and monitoring of authentication functions to detect attacks and failures in real time.

Important requirements:

- Ensure that all authentication failures are logged and reviewed.
- Ensure that all password failures are logged and reviewed.
- Ensure that all account lockouts are logged and reviewed.

Logging and monitoring help identify suspicious authentication activity.

---

6. Use Authentication Protocols That Require No Password

Username, password, and MFA authentication is generally secure, but it is not always the best or safest option.

For example, third-party applications may need to connect to a web application from:

- Mobile devices.
- Another website.
- Desktop applications.
- Other external systems.

It is **NOT safe** to allow third-party applications to store users' username/password combinations.

Why?

- The third-party application gains access to users' credentials.
- The attack surface expands beyond the original application.
- The original application loses control over how those credentials are stored and protected.

Passwordless authentication protocols can help avoid exposing user credentials to third-party applications.

6.1 OAuth 2.0 and OAuth 2.1

OAuth is an **Authorization Framework** for delegated access to APIs.

Important distinction:

- OAuth is used for authorization.
- It allows applications to access resources on behalf of users without directly sharing their passwords.

OAuth 2.1 Note:

- OAuth 2.1 is an active IETF Working Group draft.
- It consolidates OAuth 2.0 and widely adopted best practices.
- It is intended to replace RFC 6749/6750.
- Because Internet-Drafts are revisioned, the unversioned OAuth 2.1 Datatracker page should be used for the current draft.

6.2 OpenID Connect (OIDC)

OpenID Connect 1.0 (OIDC) is an identity layer built on top of OAuth.

It defines how a client, called the **Relying Party**, verifies the identity of the end user using an **ID Token**, which is a signed JWT.

It also defines how to obtain user claims in an interoperable way.

Important distinction:

Technology	Purpose
OAuth	Authorization to APIs.
OpenID Connect (OIDC)	Authentication and Single Sign-On (SSO).

OIDC Implementation Guidance

1. **Validate ID Tokens**
    - Validate the issuer (`iss`).
    - Validate the audience (`aud`).
    - Validate the signature using the provider's JWKs.
    - Validate expiration (`exp`).
2. **Use Well-Maintained Libraries**
    - Prefer well-maintained libraries and SDKs.
    - Use provider discovery and JWKS endpoints.
3. **UserInfo Endpoint**
    - Use the UserInfo endpoint when additional claims beyond the ID Token are required.

OpenID 2.0 vs OpenID Connect

- OpenID 2.0, also known as OpenID, was a separate legacy authentication protocol.
- It has been superseded by OpenID Connect.
- OpenID 2.0 is considered obsolete.
- New systems should not implement OpenID 2.0.

6.3 Secure Federated Account Linking

When linking an OIDC identity to an existing application account, follow these recommendations:

1. Identify the federated identity using the combination of `iss` (issuer) and `sub` (subject), rather than `sub` alone.
2. Do not automatically link accounts just because their `email`, `preferred_username`, or other profile attributes match.
    - These claims are not guaranteed to be unique or stable.
3. Require an authenticated session with the existing application account before adding or removing linked identities.
4. Authenticate the identity being added and validate the resulting ID Token before saving the link.
5. Bind the authentication response to the user's linking session using OAuth 2.0 protections.
6. After unlinking an identity, prevent access to the application account through that removed identity.
7. Notify users when an identity is added or when an identity is removed without terminating the application account.

6.4 SAML

SAML (Security Assertion Markup Language) is often considered a competitor to OpenID.

- The most recommended version is **SAML 2.0**.
- It is feature-complete and provides strong security.
- Like OpenID, SAML uses Identity Providers.
- Unlike OpenID, SAML is XML-based and provides more flexibility.
- SAML is based on browser redirects that send XML data.
- Authentication can be initiated by either the Service Provider or the Identity Provider.

How SAML Supports SSO

SAML allows users to navigate between different portals while remaining authenticated without repeatedly logging in.

SAML Usage

- OpenID has taken most of the consumer market.

- SAML is often preferred for enterprise applications.
- It is commonly used in intranet websites.
- It is frequently used when enterprise federation is required for web services and web applications.

Examples include:

- SAP ERP.
- SharePoint using Active Directory Federation Services 2.0.

6.5 FIDO

The **Fast Identity Online (FIDO) Alliance** created two protocols to facilitate online authentication:

1. Universal Authentication Framework (UAF).
2. Universal Second Factor (U2F).

Both protocols use a **public-key cryptography challenge-response model**.

A. Universal Authentication Framework (UAF)

UAF focuses on passwordless authentication.

It uses existing security technologies available on devices, including:

- Fingerprint sensors.
- Cameras for face biometrics.
- Microphones for voice biometrics.
- Trusted Execution Environments (TEEs).
- Secure Elements (SEs).

UAF is designed to integrate device capabilities into a common authentication framework.

It works with:

- Native applications.
- Web applications.

B. Universal Second Factor (U2F)

U2F adds a second factor to existing password-based authentication.

- Uses a hardware token, typically USB.
- The token stores cryptographic authentication keys.
- The token uses those keys for signing.
- The same token can be used as a second factor for multiple applications.

- U2F works with web applications.

Phishing Protection:

U2F provides protection against phishing by using the website URL to look up the stored authentication key.

C. FIDO2

FIDO2 combines:

- WebAuthn.
- Client-to-Authenticator Protocols (CTAP).

Important points:

- U2F is now named CTAP1.
- UAF is a separate protocol.
- WebAuthn credentials form the foundation of modern Passkeys technology.
- Passkeys allow users to log in securely using local user verification, such as biometrics or a device PIN.
- Passkeys often support credential synchronization across devices.

6.6 Hardware-Backed Key Storage

For many authenticators, including common platform passkeys, the private key is generated and stored by the operating system's secure key manager.

Depending on the platform, keys may be protected by:

- **TPM:** Trusted Platform Module on Windows.
- **Secure Enclave:** On Apple devices.
- **Android Keystore / StrongBox:** On Android devices.
- Other software-based mechanisms.

Important Security Considerations

- Private keys are generally intended to be non-exportable and bound to the authenticator.
- The platform security module signs a server challenge using the private key.
- Some authenticators support credential synchronization or backups that may involve export or server-side storage.
- Not all implementations are hardware-backed.
- Relying parties should not assume that keys are hardware-backed and non-exportable unless this has been verified through authenticator properties or attestation.

---

7. Password Managers

Password managers are programs, browser plugins, or web services that automate the management of multiple credentials.

They help users enter passwords on websites in two common ways:

1. Pasting passwords into login forms.
2. Simulating the user typing passwords into login forms.

Recommendations for Web Applications

Web applications should not make password managers difficult to use.

Follow these recommendations:

1. Use standard HTML forms for username and password input with appropriate `type` attributes.
2. Avoid plugin-based login pages, such as Flash or Silverlight.
3. Implement a reasonable maximum password length of at least **64 characters**.
4. Allow all printable characters in passwords.
5. Allow users to paste into:
    - Username fields.
    - Password fields.
    - MFA fields.
6. Allow users to navigate between username and password fields using a single press of the `Tab` key.

---

8. Changing a User's Registered Email Address

Users may need to change their registered email addresses.

The recommended process depends on whether the user has MFA enabled.

Important note: The process is less stringent when MFA is enabled because proof of identity is stronger than relying solely on a password.

8.1 Recommended Process If the User HAS MFA Enabled

1. Confirm the validity of the user's authentication cookie/token.
    - If it is not valid, display a login screen.
2. Explain the email-change process to the user.

3. Ask the user to submit a proposed new email address.
    - Ensure that it complies with system rules.
4. Request MFA for identity verification.
5. Store the proposed new email address as a pending change.
6. Create and store **two time-limited nonces**:
    - One for system administrators' notification.
    - One for user confirmation.
7. Send two email messages containing links with those nonces:
    - **Notification-only email to the current address:**
        - Alerts the user about the upcoming email change.
        - Provides a link to report unexpected activity.
    - **Confirmation-required email to the proposed new address:**
        - Instructs the user to confirm the change.
        - Provides a link for unexpected situations.
8. Handle responses from the links accordingly.

8.2 Recommended Process If the User DOES NOT HAVE MFA Enabled

1. Confirm the validity of the user's authentication cookie/token.
    - If it is not valid, display a login screen.
2. Explain the email-change process to the user.
3. Ask the user to submit a proposed new email address.
    - Ensure that it complies with system rules.
4. Request the user's current password for identity verification.
5. Store the proposed new email address as a pending change.
6. Create and store **three time-limited nonces**:
    - One for system administrators' notification.
    - One for user confirmation.
    - One for an additional step for password reliance.
7. Send two email messages containing links to those nonces:
    - **Confirmation-required email to the current address:**
        - Instructs the user to confirm the change.
        - Provides a link for an unexpected situation.
    - **Separate confirmation-required email to the proposed new address:**
        - Instructs the user to confirm the change.
        - Provides a link for unexpected situations.
8. Handle responses from the links accordingly.

8.3 Notes on the Above Processes

- Google follows a different approach for accounts secured only by passwords.
- In Google's approach, the current email address receives a notification-only email.
- This approach carries risks and requires user vigilance.
- Regular social engineering training is crucial.

- System administrators and help desk staff should be trained to:
    - Follow the prescribed process.
    - Recognize social engineering attacks.
    - Respond appropriately to such attacks.

---

9. Adaptive or Risk-Based Authentication

Adaptive or Risk-Based Authentication is a feature of advanced applications that allows different authentication stages depending on environmental and contextual attributes.

These attributes may include:

- Sensitivity of the data being accessed.
- Time of day.
- User location.
- IP address.
- Device fingerprint.

Examples

Example 1: Device-Based Authentication

- An application may require MFA for the first login from a particular device.
- Subsequent logins from that device may not require the same additional step.

Example 2: Sensitive Account Access

- A Single Sign-On solution may authenticate a user and allow them to remain logged in for a day.
- However, re-authentication may be required when the user attempts to access their profile page.

Important Security Recommendation

Use device fingerprints, IP addresses, and remembered-device cookies as **risk signals**, not as substitutes for authentication.

- Require authentication or a valid authenticated session before exposing private account data.
- An authenticated session secret is different from device and browser characteristics used for monitoring.

Example: Banking Application

- A banking application may allow users to view their account balance within an authenticated session.
- It may require stronger or more recent authentication for transaction details or money movement.

Questions to Consider When Implementing Risk-Based Authentication

1. Are the policies aligned with corporate policies and regulatory requirements?
2. Which user or device attributes will be monitored at session start?
    - IP address.
    - Geolocation.
    - Device fingerprint.
    - Time of day.
    - Behavioral biometrics.
3. Which signals need to be refreshed during an active session, and how frequently?
4. How will the application ensure signal accuracy and handle missing or low-confidence data?
5. What scoring model will convert raw signals into a risk tier?
    - Weights.
    - Thresholds.
    - Machine Learning.
    - Rule-based models.
    - Hybrid models.
6. Where will the risk model run?
    - Edge.
    - API Gateway.
    - Central service.
7. What is the latency budget?
8. What action should be taken for each risk tier?
    - Allow.
    - CAPTCHA.
    - Step-up MFA.
    - Block.
    - Revoke session.
9. What user-facing messages and error codes should accompany each action?
10. At which code or platform layers should the risk engine be invoked?
    - Login controller.
    - Middleware.
    - API Gateway.
    - Service mesh.
11. How will decisions be propagated consistently across web, mobile, and API clients?
12. How will tokens and cookies be modified, extended, or revoked when a mid-session risk check escalates?
13. How will state be synchronized across multiple concurrent devices or browser tabs?

14. What monitoring and alerting mechanisms will detect suspicious activity, and how will users be notified?

Quick Revision: Important Concepts

Concept	Key Point
Authentication (AuthN)	Verifies the identity of a user or entity.
Digital Identity	Unique representation of a subject in an online transaction.
Identity Proofing	Establishes that a subject is who they claim to be.
Session Management	Maintains user state through a session identifier.
User ID	Should ideally be randomly generated.
Password Length	Minimum 8 characters with MFA; 15 without MFA.
Maximum Password Length	At least 64 characters.
Password Composition	Do not impose character-type restrictions.
Password Rotation	Avoid arbitrary periodic changes; rotate after compromise.
Password Comparison	Use secure functions and constant-time comparisons.
TLS	Protects credentials and authenticated sessions during transmission.
Re-authentication	Required for sensitive actions and high-risk events.
Generic Error Messages	Prevent user enumeration.
Timing Attacks	Processing-time differences may reveal valid usernames.
Brute Force	Multiple passwords tested against one account.
Credential Stuffing	Reusing leaked username/password combinations.
Password Spraying	One weak password tested against many accounts.
MFA	Strong defense against password-related attacks.
Login Throttling	Limits repeated login attempts.

Account Lockout	Temporarily prevents login after repeated failures.
CAPTCHA	Defense-in-depth against automated attempts.
OAuth	Authorization framework for delegated API access.
OIDC	Authentication and SSO layer built on OAuth.
SAML	XML-based enterprise federation and SSO protocol.
FIDO	Public-key-based authentication protocols.
UAF	Passwordless authentication.
U2F	Hardware-based second factor.
FIDO2	Combines WebAuthn and CTAP.
Passkeys	Use WebAuthn credentials and local user verification.
Password Managers	Should be supported through standard forms and paste functionality.
Risk-Based Authentication	Adjusts authentication requirements according to risk signals.